Applied Deep Learning with TensorFlow, part 1/2

© Mira Vorne, LAB University of Applied Sciences

TensorFlow and Tensors, The Numerical Foundation of Deep Learning

Deep learning frameworks such as TensorFlow are designed to perform large-scale numerical computation efficiently. At the core of these computations lies the tensor. A tensor is a general mathematical object used to represent numerical data of arbitrary dimensionality. Scalars, vectors, matrices, and higher-dimensional arrays are all specific cases of tensors.

A scalar is a tensor with zero dimensions and represents a single number. A vector is a one-dimensional tensor and can represent quantities such as coordinates or feature lists. A matrix is a two-dimensional tensor, commonly used to represent tabular data or pixel grids. More generally, a tensor with more than two dimensions can represent complex data such as color images, video frames, or batches of samples processed together.

TensorFlow provides two main ways of creating tensors: tf.constant and tf.Variable. Constants represent immutable values, while variables allow their values to change. In practice, model parameters such as weights and biases are stored internally as variables, while data inputs are typically treated as constants. Most users do not need to decide this manually, as TensorFlow manages these details automatically.

Each tensor has several important properties:

- Shape, which describes how many elements exist along each dimension
- Rank (number of dimensions), which indicates tensor complexity
- Data type, which affects memory usage and numerical precision

Understanding these properties is essential because shape mismatches are one of the most common sources of errors in deep learning workflows.

TensorFlow also supports vectorized mathematical operations such as addition, multiplication, matrix multiplication, and aggregation. Matrix multiplication is central to neural networks. It allows inputs to be transformed through learned weight matrices, forming the basis of linear layers.

Finally, TensorFlow integrates seamlessly with NumPy. Tensors can be converted to and from NumPy arrays, allowing users to combine traditional numerical Python workflows with GPU-accelerated deep learning computation. A key advantage of TensorFlow tensors is their ability to run efficiently on GPUs and specialized hardware such as TPUs.

Tensors and Numerical Foundations

From Numbers to Tensors: Why Scalars, Vectors, and Matrices Matter

Consider a simple real-world situation: collecting information about apartments.

Suppose we record only the price of each apartment.

A single apartment price such as 250000 is represented as a scalar tensor.

It contains one numerical value and no structure.

If we now describe one apartment using several features—price, size, and number of rooms, these values form a vector:

```
[250000, 55, 2]
```

This vector represents one object with multiple attributes.

Now suppose we collect this information for many apartments. The data naturally becomes a matrix, where:

- Each row represents one apartment
- Each column represents one feature

This structured numerical representation allows machine learning models to detect patterns such as how apartment size affects price.

Key idea:

Neural networks do not “see apartments” or “know features.”

They operate entirely on numerical tensors whose shape encodes meaning.

Why Shape Errors Are So Common

Imagine a model that expects data shaped like:

```
(number of apartments, 3 features)
```

If data accidentally arrives shaped like:

```
(3 features, number of apartments)
```

then mathematically the data contains the same numbers, but the meaning is reversed.

TensorFlow cannot guess intent, so it raises an error.

This explains why developers spend much of their time checking:

.shape

.ndim

alignment between inputs and weights

Key idea

Most deep learning bugs are data-structure bugs, not mathematical ones.

Matrix Multiplication as Feature Transformation

In neural networks, a dense layer performs matrix multiplication:

```
output = inputs * weights + bias
```

If inputs represent apartment features and weights represent how strongly each feature influences price, then matrix multiplication applies learned importance to each feature.

This turns raw data into learned representations, which subsequent layers refine further.

Key idea

Learning means adjusting weight matrices so that numerical transformations align with target outputs.

CODING, START HERE: Tensor and numerical foundations (TensorFlow)

```
import tensorflow as tf

import numpy as np


print("TF:", tf.__version__)


# 1) Scalars, vectors, matrices, tensors

scalar = tf.constant(7)  # rank-0

vector = tf.constant([10, 20, 30]) # rank-1

matrix = tf.constant([[1., 2.], [3., 4.]])  # rank-2

tensor  = tf.constant([[[1],[2]], [[3],[4]]]) # rank-3 (2x2x1)


print(scalar.shape, vector.shape, matrix.shape, tensor.shape)


# 2) Dtypes and casting

x = tf.constant([1.7, 7.4]) # float32 default

y = tf.constant([7, 10]) # int32 default

x16 = tf.cast(x, tf.float16) # reduced precision floats

y32 = tf.cast(y, tf.float32) # cast ints → floats


# 3) Indexing and shaping

rank2 = tf.constant([[10, 7], [3, 4]])

last_col = rank2[:, -1] # column slice

expanded = tf.expand_dims(rank2, -1) # add channel-like axis
```

```
squeezed = tf.squeeze(expanded, axis=-1)


# 4) Broadcasting & basic ops
A = tf.constant([[10., 7.], [3., 4.]])
print("A+10:\n", A + 10)
print("A*2:\n", A * 2)


# 5) Matrix multiplication rules (inner dims match)
X = tf.constant([[1, 2], [3, 4], [5, 6]], dtype=tf.float32)  # (3x2)
Y = tf.constant([[7, 8], [9, 10], [11, 12]], dtype=tf.float32) # (3x2)
XtY = tf.matmul(X, tf.transpose(Y))    # (3x2) @ (2x3) -> (3x3)


# 6) Aggregations
E = tf.constant(np.random.randint(0, 100, size=10))
print("min:", tf.reduce_min(E).numpy(),
    "max:", tf.reduce_max(E).numpy(),
    "mean:", tf.reduce_mean(tf.cast(E, tf.float32)).numpy(),
    "sum:", tf.reduce_sum(E).numpy())


# 7) One-hot encoding
idx = tf.constant([0, 1, 2, 3])
oh  = tf.one_hot(idx, depth=4)


# 8) NumPy interop
arr = np.array([3., 7., 10.])
t   = tf.constant(arr) # NumPy -> Tensor
back_to_np = t.numpy() # Tensor -> NumPy


print("OK")
```

Why this matters: You'll debug far more shape/dtype issues than math issues. Building intuition here prevents 80% of downstream errors.

Neural Network Regression, Predicting Continuous Values

Regression problems involve predicting numerical values. Examples include estimating house prices, predicting medical costs, or forecasting physical measurements. In neural network regression, inputs are numerical features and the output is a continuous number.

The simplest regression neural network consists of:

- An input layer, receiving feature values
- One or more hidden layers, which learn intermediate representations
- An output layer, producing the final numerical prediction

For regression, the output layer typically contains a single neuron without a restrictive activation function. Loss functions such as Mean Absolute Error (MAE) or Mean Squared Error (MSE) are used to quantify prediction error.

Model training follows a consistent workflow:

- **Create the model architecture**
- **Compile the model using a loss function and optimizer**
- **Fit the model to training data**
- **Evaluate the model on unseen test data**

Evaluation must be performed on data that was not used during training. This separation ensures that performance reflects a model's ability to generalize rather than merely memorize training examples.

Visualizing regression predictions is particularly valuable. Plotting predicted values alongside true values provides intuitive insight into how well the model has learned underlying relationships.

As model complexity increases, issues such as overfitting can arise. Overfitting occurs when a model learns the training data too precisely and fails to generalize to new inputs. Balancing model complexity, training duration, and data preprocessing is therefore essential.

Regression with Neural Networks

Learning a Linear Relationship

Consider a dataset where:

```
y = x + 10
```

Humans immediately recognize this pattern.

A neural network does not.

The model starts with random weights and gradually updates them to reduce prediction error. After enough iterations, the learned weights approximate the correct linear slope and offset.

The network has not memorized numbers.

It has learned a general rule that applies to unseen inputs.

Key idea

Regression networks learn functions, not datasets.

Why More Training Sometimes Makes Things Worse

If we train longer and longer on the same small dataset, prediction error on training data keeps shrinking.

However, predictions on unseen data may worsen.

This occurs because the model starts fitting noise rather than signal.

This example introduces overfitting, which appears whenever:

- Models are too complex
- Data is limited
- Training runs too long

Key idea

Good performance means generalization, not perfection on training data.

Visual Evaluation Beats Numbers Alone

Two models may have similar average error, yet one makes systematically worse mistakes.

By plotting predictions versus ground truth:

- We see whether errors are random
- We detect bias and pattern mismatches

Visualization transforms abstract metrics into intuition.

Key idea

Numbers summarize performance; plots explain it.

CODING, START HERE: Regression (Neural Network predicting a number)

Task: Learn y=x+10 and generalize to new inputs.

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt


# Synthetic linear data

X = np.arange(-20, 21, 2, dtype=np.float32)  # [-20, -18, ..., 20]

y = X + 10


# Train/test split (simple)
```

```
X_train, y_train = X[:20], y[:20]
X_test,  y_test  = X[20:], y[20:]

# 1) CREATE
model = tf.keras.Sequential([
   tf.keras.layers.Input(shape=(1,)),
   tf.keras.layers.Dense(1)  # linear output for regression
])

# 2) COMPILE
model.compile(
   loss="mae",
   optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
   metrics=["mae"]
)

# 3) FIT
history = model.fit(X_train, y_train, epochs=200, verbose=0)

# Evaluate
loss, mae = model.evaluate(X_test, y_test, verbose=0)
print("Test MAE:", mae)

# Predict and visualize
y_pred = model.predict(X_test).squeeze()

plt.figure(figsize=(6,4))
plt.scatter(X_train, y_train, c='b', label='Train')
plt.scatter(X_test,  y_test,  c='g', label='Test')
plt.scatter(X_test,  y_pred,  c='r', label='Pred')
```

```
plt.legend(); plt.title("Regression: y = x + 10 learned")

plt.show()
```

Why this matters: The workflow (Create → Compile → Fit) is identical for all Keras models. What changes is the loss, last layer, and data.

Neural network classification, predicting categories

Classification problems differ fundamentally from regression in that the goal is to predict classes rather than numbers. These classes may be binary (e.g., yes/no) or multiclass (e.g., identifying object types).

In binary classification, the output layer contains one neuron with a **sigmoid** activation function, which outputs probabilities between 0 and 1. In multiclass classification, the output layer has one neuron per class and uses **softmax** activation to produce a probability distribution over classes.

The choice of loss function reflects the task:

- Binary classification uses binary cross-entropy
- Multiclass classification uses categorical or sparse categorical cross-entropy

A critical concept introduced in classification is non-linearity. Linear models cannot solve problems where classes are not separable by straight lines. Activation functions such as ReLU and sigmoid allow networks to learn curved decision boundaries.

Visualization plays a central role in understanding classification models. Plotting decision boundaries reveals whether a model's predictions align with the structure of the data.

Evaluation extends beyond accuracy. Metrics such as precision and confusion matrices provide deeper insight into predictive performance and error patterns.

For image-based classification, data normalization significantly improves training dynamics. Scaling pixel values into the range [0, 1] stabilizes gradient updates and accelerates convergence.

Why Linear Models Fail on Circular Data

In the circle dataset:

- Class 0 is inside
- Class 1 is outside

Any straight line fails to separate classes.

Adding more layers without non-linear activation still produces straight boundaries.

Only when non-linear activations are used does the network become capable of drawing curved boundaries.

Key idea

Depth without non-linearity adds no expressive power.

Probability vs. Class Labels

A binary classification model outputs values like:

```
0.93
```

This is not a label.

It is a probability.

We interpret:

- values above 0.5 > Class 1
- values below 0.5 > Class 0

Understanding this helps explain:

- Threshold tuning
- False positives and negatives
- Confusion matrices

Key idea

Classification models reason in probabilities, not categories.

Why Normalization Improves Learning

Fashion MNIST pixel values range from 0 to 255.

Without normalization:

- Gradients fluctuate wildly
- Learning slows
- Accuracy suffers

After scaling values to [0, 1]:

- Weight updates stabilize
- Training accelerates
- Validation accuracy increases

Key idea

Same model + better numerical input > dramatically better learning.

CODING, START HERE: Classification (Binary + Multiclass)

Binary classification on non-linear data (circles)

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import make_circles
```

```
from sklearn.model_selection import train_test_split

# Non-linear separable data
X, y = make_circles(n_samples=1000, noise=0.03, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# 1) CREATE (use non-linear activations + sigmoid output)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

# 2) COMPILE
model.compile(
    loss="binary_crossentropy",
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
    metrics=["accuracy"]
)

# 3) FIT
history = model.fit(X_train, y_train, epochs=25, verbose=0)

# Evaluate
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {acc:.3f}")

# Visualize decision boundary
```

```
xx, yy = np.meshgrid(np.linspace(X[:,0].min()-0.1, X[:,0].max()+0.1, 200),
                     np.linspace(X[:,1].min()-0.1, X[:,1].max()+0.1, 200))
grid = np.c_[xx.ravel(), yy.ravel()]
probs = model.predict(grid, verbose=0).reshape(xx.shape)

plt.figure(figsize=(6,5))
plt.contourf(xx, yy, (probs>0.5).astype(int), cmap=plt.cm.RdYlBu, alpha=0.6)
plt.scatter(X_test[:,0], X_test[:,1], c=y_test, cmap=plt.cm.RdYlBu, edgecolor='k')
plt.title("Binary classification decision boundary (non-linear)")
plt.show()
```

Key: Non-linear activations (e.g., ReLU) + sigmoid output + binary cross-entropy.

Multiclass classification with Fashion-MNIST

```
import tensorflow as tf
import matplotlib.pyplot as plt

# Load Fashion-MNIST
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

# Normalize to [0,1]
x_train = x_train.astype("float32")/255.0
x_test  = x_test.astype("float32")/255.0

# 1) CREATE (flatten > Dense ReLU > Dense softmax)
model = tf.keras.Sequential([
   tf.keras.layers.Input(shape=(28, 28)),
   tf.keras.layers.Flatten(),
   tf.keras.layers.Dense(128, activation="relu"),
   tf.keras.layers.Dense(10, activation="softmax")
])
```

```
# 2) COMPILE
model.compile(
  loss="sparse_categorical_crossentropy",  # labels are ints, not one-hot
  optimizer="adam",
  metrics=["accuracy"]
)

# 3) FIT
history = model.fit(x_train, y_train, epochs=10, validation_split=0.1, verbose=0)

# Evaluate
loss, acc = model.evaluate(x_test, y_test, verbose=0)
print("Test accuracy:", acc)

# Show a few preds
preds = model.predict(x_test[:8]).argmax(axis=1)
plt.figure(figsize=(10,3))
for i in range(8):
  plt.subplot(1,8,i+1)
  plt.imshow(x_test[i], cmap='gray')
  plt.title(int(preds[i]))
  plt.axis('off')
plt.suptitle("Predicted class indices"); plt.show()
```

Key: For multiclass, end with softmax and use categorical loss (sparse or one-hot version depending on labels).

Convolutional Neural Networks, Learning from Images

Traditional dense layers treat inputs as flat vectors, ignoring spatial structure. Convolutional Neural Networks (CNNs) address this limitation by processing images using spatial filters that learn local features.

A CNN typically consists of:

- Convolutional layers, which extract features such as edges, shapes, and textures

- Activation functions, introducing non-linearity
- Pooling layers, which reduce spatial resolution while preserving important information
- Fully connected layers, which perform final classification

CNNs are particularly effective because they use parameter sharing. Filters are applied across entire images, allowing models to detect patterns regardless of position.

Overfitting is a common challenge in CNNs. Techniques such as max pooling and data augmentation help models generalize more effectively. Data augmentation artificially increases dataset diversity by rotating, flipping, or scaling images during training.

CNN-based image classifiers significantly outperform traditional dense networks when applied to visual data, even with fewer trainable parameters.

Convolutional Neural Networks

Why Dense Networks Struggle with Images

A flattened 224×224 image produces over 150,000 inputs.

Each pixel connects to every neuron.

This destroys spatial relationships:

- Edges
- Shapes
- Position

CNNs preserve spatial structure by using small filters that move across the image.

Key idea

CNNs learn local patterns first, then assemble them into larger concepts.

What a Convolutional Filter Learns

A 3×3 filter might activate strongly on:

- Horizontal edges
- Bright-dark transitions
- Corners

Each filter learns one kind of visual pattern.

Multiple filters create a feature map, allowing the model to see many properties at once.

Key idea

Convolution replaces brute-force connections with intelligent feature extraction.

Why Pooling Helps Generalization

Pooling reduces resolution by keeping only the strongest activations.

This makes the model:

- Less sensitive to noise

- More position-invariant

A pizza slightly off-center is still recognized as pizza.

Key idea

Pooling trades detail for robustness.

Data Augmentation as Realism Injection

Real-world photos vary in:

- Angle
- Lighting
- Position

Data augmentation simulates these variations.

The model learns concepts, not camera conditions.

Key idea

Augmentation teaches tolerance to variability.

CODING, START HERE: CNNs for Images

Task: Binary image classification with a small CNN on a two-class subset built from Fashion-MNIST (just for demo).

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt


# Use Fashion-MNIST but map 2 classes to a binary task (e.g., class 9=Ankle boot vs 7=Sneaker)

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

# Keep only classes 7 and 9

train_mask = np.isin(y_train, [7, 9])

test_mask  = np.isin(y_test,  [7, 9])


x_train = x_train[train_mask][..., None]/255.0

y_train = (y_train[train_mask] == 9).astype(np.float32)  # 1 if class 9 else 0

x_test  = x_test[test_mask][..., None]/255.0

y_test  = (y_test[test_mask] == 9).astype(np.float32)
```

```
# 1) CREATE (small CNN)
model = tf.keras.Sequential([
  tf.keras.layers.Input(shape=(28,28,1)),
  tf.keras.layers.Conv2D(16, 3, activation="relu"),
  tf.keras.layers.MaxPool2D(),
  tf.keras.layers.Conv2D(32, 3, activation="relu"),
  tf.keras.layers.MaxPool2D(),
  tf.keras.layers.Flatten(),
  tf.keras.layers.Dense(64, activation="relu"),
  tf.keras.layers.Dense(1, activation="sigmoid")
])

# 2) COMPILE
model.compile(
  loss="binary_crossentropy",
  optimizer="adam",
  metrics=["accuracy"]
)

# 3) FIT
history = model.fit(x_train, y_train, epochs=5, validation_split=0.1, verbose=0)

# Evaluate
loss, acc = model.evaluate(x_test, y_test, verbose=0)
print("Test accuracy:", acc)
```

Why CNN: Convolutions preserve spatial structure and learn local patterns that combine into larger features, which dense layers on flattened pixels cannot capture as effectively.

Transfer Learning, Leveraging Pretrained Models

Transfer learning addresses one of the biggest challenges in deep learning: the need for large datasets. Instead of training models from scratch, transfer learning reuses patterns learned by models trained on large datasets such as ImageNet.

Two main strategies exist:

- Feature extraction, where pretrained layers are frozen and used as fixed feature generators
- Fine-tuning, where selected pretrained layers are retrained on new data

Feature extraction is particularly effective when training data is limited. By using models such as ResNet50 or EfficientNet, it is possible to achieve strong performance even with a fraction of the original dataset size.

TensorFlow Hub simplifies transfer learning by providing access to pretrained models as modular components. By attaching a new output layer, pretrained feature extractors can be adapted quickly to new classification tasks.

Experiment tracking becomes increasingly important as model complexity grows. Tools such as TensorBoard allow practitioners to compare training runs, visualize learning curves, and diagnose performance issues.

Transfer learning demonstrates a key principle of modern deep learning: knowledge learned on one problem can often be reused on another, dramatically reducing cost, data requirements, and training time.

Across these five chapters, a unified deep learning workflow emerges:

- Represent data numerically using tensors
- Select appropriate model architectures for the task
- Train models using gradient-based optimization
- Evaluate performance using appropriate metrics
- Improve results through architecture design, preprocessing, and transfer learning

Deep learning is inherently experimental. Progress comes through iteration, visualization, and careful evaluation. The techniques presented here form a foundation upon which more advanced methods, such as fine-tuning, regularization strategies, and large-scale model deployment, can be built.

Transfer Learning

Why Pretrained Models Know Useful Patterns

Models trained on ImageNet have already learned:

- Edges
- Textures
- Shapes

These features appear in almost all images, including food.

By reusing them, we skip years of prior learning.

Key idea

Visual features generalize extremely well.

Feature Extraction with Frozen Weights

By freezing a pretrained network:

- We preserve learned features
- We train only a small classification head

This prevents overfitting and speeds up training.

The model adapts interpretation, not perception.

Key idea

Feature extraction separates seeing from deciding.

Why EfficientNet Beats Larger Models

EfficientNet achieves higher accuracy with fewer parameters.

This shows:

- Performance is not proportional to size
- Architectural efficiency matters

Key insight

Better design often beats brute force.

Comparing Experiments with TensorBoard

By comparing training curves:

- We see convergence speed
- We detect overfitting
- We identify the most stable model

Experiment tracking turns intuition into evidence.

Key idea

Modern deep learning is experimental science.

Final Perspective

These examples collectively demonstrate how:

- Numerical structure enables learning
- Architecture must match data
- Visualization reveals understanding
- Transfer learning multiplies efficiency

Together, they form a complete, applied mental model of deep learning with TensorFlow.

CODING, START HERE: Transfer Learning (Feature Extraction)

```
import tensorflow as tf

from tensorflow.keras import layers, models



# Data
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Keep classes 0,1,2 only
mask_train = (y_train < 3).flatten()
mask_test  = (y_test  < 3).flatten()
x_train = x_train[mask_train].astype("float32")/255.0
y_train = y_train[mask_train].flatten()
x_test  = x_test[mask_test].astype("float32")/255.0
y_test  = y_test[mask_test].flatten()

# Small CNN model
model = models.Sequential([
  layers.Conv2D(32, (3,3), activation="relu", padding="same", input_shape=(32,32,3)),
  layers.MaxPooling2D(),

  layers.Conv2D(64, (3,3), activation="relu", padding="same"),
  layers.MaxPooling2D(),

  layers.Conv2D(128, (3,3), activation="relu", padding="same"),
  layers.MaxPooling2D(),

  layers.Flatten(),
  layers.Dense(64, activation="relu"),
  layers.Dense(3, activation="softmax")
])

model.compile(
  optimizer="adam",
  loss="sparse_categorical_crossentropy",
  metrics=["accuracy"]
```

```
)


# Train
history = model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1,
    verbose=1
)


# Evaluate
loss, acc = model.evaluate(x_test, y_test, verbose=0)
print("Test accuracy (tiny CNN):", acc)
```

Why this works: The base has already learned useful visual features on ImageNet; freezing it and training a small head often yields strong accuracy even with fewer task-specific images.

The 3 Keras Training Steps

Create (Build)

Define the layers and the computation graph (e.g., tf.keras.Sequential([...]) or Functional API).

Compile

Configure the learning process: loss, optimizer, and metrics.

Fit

Run gradient-based learning on your data: model.fit(X, y, ...).

These three steps are the consistent mental model across regression, classification, CNNs, and transfer learning. You just swap the last layer, loss, and data depending on the task.